Importing Modules and Libraries

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# importing required libraries
import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt

import pickle
from os import path

from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

from sklearn import metrics
from sklearn import preprocessing
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

from sklearn.svm import SVC
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
```

Importing Datasets

```
#data = pd.read_csv('/content/drive/MyDrive/IoT_ML_Dataset/UNSW/UNSW_NB15.csv')
data = pd.read_csv('/content/drive/MyDrive/Datasets/UNSW_NB15.csv')
```

```
data.head(n=5)
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm	is_ftp_log
0	1	0.121478	tcp	-	FIN	6	4	258	172	74.087490	...	1	1	
1	2	0.649902	tcp	-	FIN	14	38	734	42014	78.473372	...	1	2	
2	3	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	...	1	3	
3	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	...	1	3	
4	5	0.449454	tcp	-	FIN	10	6	534	268	33.373826	...	1	40	

5 rows × 45 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175341 entries, 0 to 175340
Data columns (total 45 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   id                 175341 non-null  int64
 1   dur                175341 non-null  float64
 2   proto              175341 non-null  object
 3   service            175341 non-null  object
 4   state              175341 non-null  object
 5   spkts              175341 non-null  int64
 6   dpkts              175341 non-null  int64
 7   sbytes             175341 non-null  int64
```

```
 8   dbytes             175341 non-null  int64
 9   rate               175341 non-null  float64
 10  sttl               175341 non-null  int64
 11  dttl               175341 non-null  int64
 12  sload              175341 non-null  float64
 13  dload              175341 non-null  float64
 14  sloss              175341 non-null  int64
 15  dloss              175341 non-null  int64
 16  sinpkt             175341 non-null  float64
 17  dinpkt             175341 non-null  float64
 18  sjit               175341 non-null  float64
 19  djit               175341 non-null  float64
 20  swin               175341 non-null  int64
 21  stcpb              175341 non-null  int64
 22  dtcpb              175341 non-null  int64
 23  dwin               175341 non-null  int64
 24  tcprtt             175341 non-null  float64
 25  synack             175341 non-null  float64
 26  ackdat             175341 non-null  float64
 27  smean              175341 non-null  int64
 28  dmean              175341 non-null  int64
 29  trans_depth        175341 non-null  int64
 30  response_body_len  175341 non-null  int64
 31  ct_srv_src         175341 non-null  int64
 32  ct_state_ttl       175341 non-null  int64
 33  ct_dst_ltm         175341 non-null  int64
 34  ct_src_dport_ltm   175341 non-null  int64
 35  ct_dst_sport_ltm   175341 non-null  int64
 36  ct_dst_src_ltm     175341 non-null  int64
 37  is_ftp_login       175341 non-null  int64
 38  ct_ftp_cmd         175341 non-null  int64
 39  ct_flw_http_mthd   175341 non-null  int64
 40  ct_src_ltm         175341 non-null  int64
 41  ct_srv_dst         175341 non-null  int64
 42  is_sm_ips_ports    175341 non-null  int64
 43  attack_cat         175341 non-null  object
 44  label              175341 non-null  int64
dtypes: float64(11), int64(30), object(4)
memory usage: 60.2+ MB
```

```
data[data['service']=='-']
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm	i
0	1	0.121478	tcp	-	FIN	6	4	258	172	74.087490	...	1	1	
1	2	0.649902	tcp	-	FIN	14	38	734	42014	78.473372	...	1	2	
2	3	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	...	1	3	
4	5	0.449454	tcp	-	FIN	10	6	534	268	33.373826	...	1	40	
5	6	0.380537	tcp	-	FIN	10	6	534	268	39.417980	...	1	40	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
175125	175126	0.653375	tcp	-	FIN	10	8	564	354	26.018748	...	1	1	
175179	175180	0.695566	tcp	-	FIN	10	8	564	354	24.440528	...	1	1	
175191	175192	0.962856	tcp	-	FIN	24	60	1256	59374	86.201883	...	1	1	
175277	175278	3.719110	tcp	-	FIN	66	340	3086	426483	108.897021	...	1	1	
175337	175338	0.505762	tcp	-	FIN	10	8	620	354	33.612649	...	1	2	

94168 rows × 45 columns

```
data['service'].replace('-',np.nan,inplace=True)
```

```
<ipython-input-7-193b95799c6a>:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chai
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  data['service'].replace('-',np.nan,inplace=True)
```

```
data.isnull().sum()
```

	0
id	0
dur	0
proto	0
service	94168
state	0
spkts	0
dpkts	0
sbytes	0
dbytes	0
rate	0
sttl	0
dttl	0
sload	0
dload	0
sloss	0
dloss	0
sinpkt	0
dinpkt	0
sjit	0
djit	0
swin	0
stcpb	0
dtcpb	0
dwin	0
tcprtt	0
synack	0
ackdat	0
smean	0
dmean	0
trans_depth	0
response_body_len	0
ct_srv_src	0
ct_state_ttl	0
ct_dst_ltm	0
ct_src_dport_ltm	0
ct_dst_sport_ltm	0
ct_dst_src_ltm	0
is_ftp_login	0
ct_ftp_cmd	0
ct_flw_http_mthd	0
ct_src_ltm	0
ct_srv_dst	0
is_sm_ips_ports	0
attack_cat	0
label	0

dtype: int64

```
data.shape
```

```
(175341, 45)
```

```
data.dropna(inplace=True)
```

```
data.shape
```

```
(81173, 45)
```

```
data['attack_cat'].value_counts()
```

attack_cat	count
Generic	39496
Normal	19488
Exploits	16187
DoS	1791
Fuzzers	1731
Reconnaissance	1703
Analysis	564
Worms	114
Backdoor	99

dtype: int64

```
data['state'].value_counts()
```

state	count
INT	39580
FIN	33640
CON	7711
REQ	228
RST	14

dtype: int64

```
data
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm
3	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	...	1	3
11	12	2.093085	tcp	smtp	FIN	62	28	56329	2212	42.520967	...	1	2
15	16	0.000002	udp	snmp	INT	2	0	138	0	500000.001300	...	1	4
17	18	0.393556	tcp	http	FIN	10	8	860	1096	43.195886	...	1	2
21	22	0.338017	tcp	http	FIN	10	6	998	268	44.376468	...	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
175335	175336	0.000006	udp	dns	INT	2	0	114	0	166666.660800	...	17	45
175336	175337	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	13	24
175338	175339	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	3	13
175339	175340	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	14	30
175340	175341	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	16	30

81173 rows × 45 columns

```
#features = pd.read_csv('/content/drive/MyDrive/IoT_ML_Dataset/UNSW/UNSW_NB15_features.csv')
features = pd.read_csv('/content/drive/MyDrive/Datasets/UNSW_NB15.csv')
```

```
features.head()
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm	is_ftp_log
0	1	0.121478	tcp	-	FIN	6	4	258	172	74.087490	...	1	1	
1	2	0.649902	tcp	-	FIN	14	38	734	42014	78.473372	...	1	2	
2	3	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	...	1	3	
3	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	...	1	3	
4	5	0.449454	tcp	-	FIN	10	6	534	268	33.373826	...	1	40	

5 rows × 45 columns

```
features['attack_cat'] = features['attack_cat'].str.lower() # This column stores the string representation of the attack type. T

# Assuming the UNSW_NB15_features.csv file contains the columns 'Name' and 'Type'
features = pd.read_csv('/content/drive/MyDrive/Datasets/UNSW_NB15_features.csv')
#features = pd.read_csv('/content/drive/MyDrive/Datasets/UNSW_NB15_features.csv') # Alternative path if the file is in a differe

# selecting column names of all data types
nominal_names = features['Name'][features['Type ']=='nominal']
integer_names = features['Name'][features['Type ']=='integer']
binary_names = features['Name'][features['Type ']=='binary']
float_names = features['Name'][features['Type ']=='float']

# selecting common column names from dataset and feature dataset
cols = data.columns
nominal_names = cols.intersection(nominal_names)
integer_names = cols.intersection(integer_names)
binary_names = cols.intersection(binary_names)
float_names = cols.intersection(float_names)

# Converting integer columns to numeric
for c in integer_names:
  pd.to_numeric(data[c])

# Converting binary columns to numeric
for c in binary_names:
  pd.to_numeric(data[c])

# Converting float columns to numeric
for c in float_names:
```

```
    pd.to_numeric(data[c])
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 81173 entries, 3 to 175340
Data columns (total 45 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 81173 non-null  int64  
 1   dur                81173 non-null  float64
 2   proto              81173 non-null  object 
 3   service            81173 non-null  object 
 4   state              81173 non-null  object 
 5   spkts              81173 non-null  int64  
 6   dpkts              81173 non-null  int64  
 7   sbytes             81173 non-null  int64  
 8   dbytes             81173 non-null  int64  
 9   rate               81173 non-null  float64
 10  sttl               81173 non-null  int64  
 11  dttl               81173 non-null  int64  
 12  sload              81173 non-null  float64
 13  dload              81173 non-null  float64
 14  sloss              81173 non-null  int64  
 15  dloss              81173 non-null  int64  
 16  sinpkt             81173 non-null  float64
 17  dinpkt             81173 non-null  float64
 18  sjit               81173 non-null  float64
 19  djit               81173 non-null  float64
 20  swin               81173 non-null  int64  
 21  stcpb              81173 non-null  int64  
 22  dtcpb              81173 non-null  int64  
 23  dwin               81173 non-null  int64  
 24  tcprtt             81173 non-null  float64
 25  synack             81173 non-null  float64
 26  ackdat             81173 non-null  float64
 27  smean              81173 non-null  int64  
 28  dmean              81173 non-null  int64  
 29  trans_depth        81173 non-null  int64  
 30  response_body_len  81173 non-null  int64  
 31  ct_srv_src         81173 non-null  int64  
 32  ct_state_ttl       81173 non-null  int64  
 33  ct_dst_ltm         81173 non-null  int64  
 34  ct_src_dport_ltm   81173 non-null  int64  
 35  ct_dst_sport_ltm   81173 non-null  int64  
 36  ct_dst_src_ltm     81173 non-null  int64  
 37  is_ftp_login       81173 non-null  int64  
 38  ct_ftp_cmd         81173 non-null  int64  
 39  ct_flw_http_mthd   81173 non-null  int64  
 40  ct_src_ltm         81173 non-null  int64  
 41  ct_srv_dst         81173 non-null  int64  
 42  is_sm_ips_ports    81173 non-null  int64  
 43  attack_cat         81173 non-null  object 
 44  label              81173 non-null  int64  
dtypes: float64(11), int64(30), object(4)
memory usage: 28.5+ MB
```

```
data
```

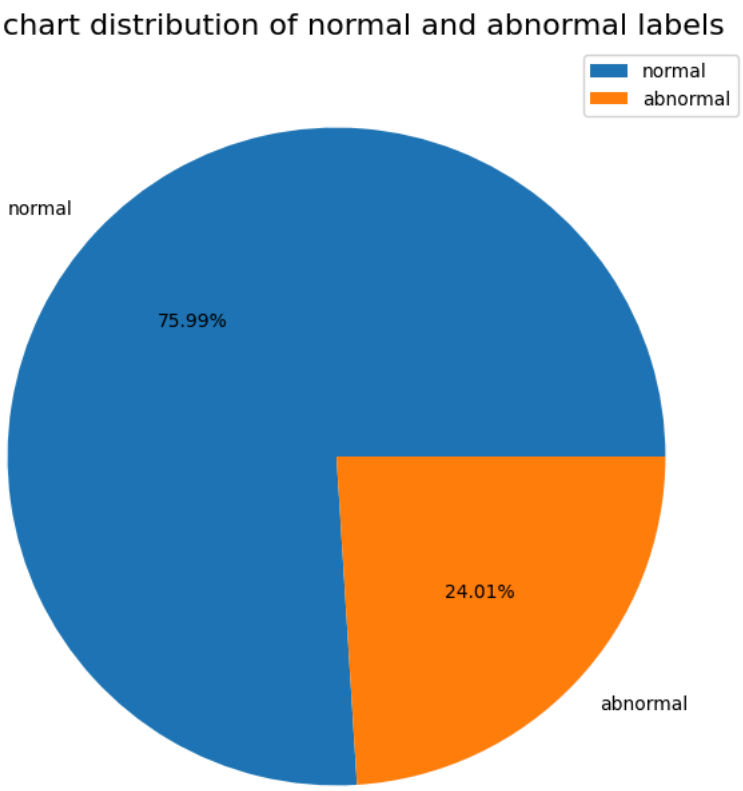
	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm
3	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	...	1	3
11	12	2.093085	tcp	smtp	FIN	62	28	56329	2212	42.520967	...	1	2
15	16	0.000002	udp	snmp	INT	2	0	138	0	500000.001300	...	1	4
17	18	0.393556	tcp	http	FIN	10	8	860	1096	43.195886	...	1	2
21	22	0.338017	tcp	http	FIN	10	6	998	268	44.376468	...	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
175335	175336	0.000006	udp	dns	INT	2	0	114	0	166666.660800	...	17	45
175336	175337	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	13	24
175338	175339	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	3	13
175339	175340	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	14	30
175340	175341	0.000009	udp	dns	INT	2	0	114	0	111111.107200	...	16	30

81173 rows × 45 columns

Data Visualization

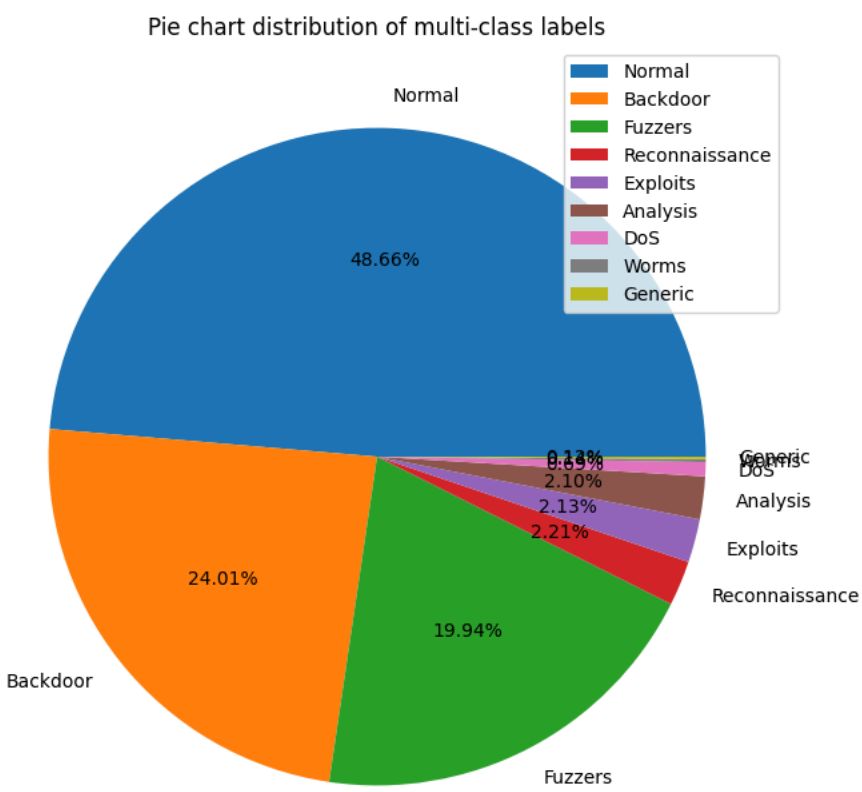
Binary Classification

```
plt.figure(figsize=(8,8))
plt.pie(data.label.value_counts(),labels=['normal','abnormal'],autopct='%0.2f%%')
plt.title("Pie chart distribution of normal and abnormal labels",fontsize=16)
plt.legend()
plt.savefig('/content/drive/MyDrive/plots/Pie_chart_binary.png')
plt.show()
```



Multi-class Classification

```
plt.figure(figsize=(8,8))
plt.pie(data.attack_cat.value_counts(),labels=data.attack_cat.unique(),autopct='%0.2f%%')
plt.title('Pie chart distribution of multi-class labels')
plt.legend(loc='best')
plt.savefig('/content/drive/MyDrive/plots/Pie_chart_multi.png')
plt.show()
```



One hot encoding

```
num_col = data.select_dtypes(include='number').columns

# selecting categorical data attributes
cat_col = data.columns.difference(num_col)
cat_col = cat_col[1:]
cat_col
```

```
Index(['proto', 'service', 'state'], dtype='object')
```

```
# creating a dataframe with only categorical attributes
data_cat = data[cat_col].copy()
data_cat.head()
```

	proto	service	state
3	tcp	ftp	FIN
11	tcp	smtp	FIN
15	udp	snmp	INT
17	tcp	http	FIN
21	tcp	http	FIN

```
# one-hot-encoding categorical attributes using pandas.get_dummies() function
data_cat = pd.get_dummies(data_cat,columns=cat_col)
```

```
data_cat.head()
```

	proto_tcp	proto_udp	service_dhcp	service_dns	service_ftp	service_ftp-data	service_http	service_irc	service_pop3	servic
3	True	False	False	False	True	False	False	False	False	
11	True	False	False	False	False	False	False	False	False	
15	False	True	False	False	False	False	False	False	False	
17	True	False	False	False	False	False	True	False	False	
21	True	False	False	False	False	False	True	False	False	

```
data.shape
```

```
(81173, 45)
```

```
data = pd.concat([data, data_cat],axis=1)
```

```
data.shape
```

```
(81173, 64)
```

```
data.drop(columns=cat_col,inplace=True)
```

```
data.shape
```

```
(81173, 61)
```

Data Normalization

```
# selecting numeric attributes columns from data
num_col = list(data.select_dtypes(include='number').columns)
num_col.remove('id')
num_col.remove('label')
print(num_col)
```

```
['dur', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt'
```

```
# using minmax scaler for normalizing data
minmax_scale = MinMaxScaler(feature_range=(0, 1))
def normalization(df,col):
  for i in col:
    arr = df[i]
    arr = np.array(arr)
    df[i] = minmax_scale.fit_transform(arr.reshape(len(arr),1))
  return df
```

```
# data before normalization
data.head()
```

	id	dur	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	...	service_radius	service_smtp	service_
3	4	1.681642	12	12	628	770	13.677108	62	252	2.740179e+03	...	False	False	
11	12	2.093085	62	28	56329	2212	42.520967	62	252	2.118251e+05	...	False	True	
15	16	0.000002	2	0	138	0	500000.001300	254	0	2.760000e+08	...	False	False	
17	18	0.393556	10	8	860	1096	43.195886	62	252	1.573347e+04	...	False	False	
21	22	0.338017	10	6	998	268	44.376468	254	252	2.127704e+04	...	False	False	

5 rows × 61 columns

```
# calling normalization() function
data = normalization(data.copy(),num_col)


# data after normalization
data.head()
```

	id	dur	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	...	service_radius	service_smtp	ser
3	4	2.802780e-02	0.001144	0.001093	0.000041	0.000053	0.000014	0.138393	0.992126	0.000001	...	False	False	
11	12	3.488529e-02	0.006344	0.002551	0.004337	0.000151	0.000043	0.138393	0.992126	0.000092	...	False	True	
15	16	1.666693e-08	0.000104	0.000000	0.000003	0.000000	0.500000	0.995536	0.000000	0.119792	...	False	False	
17	18	6.559354e-03	0.000936	0.000729	0.000059	0.000075	0.000043	0.138393	0.992126	0.000007	...	False	False	
21	22	5.633690e-03	0.000936	0.000547	0.000069	0.000018	0.000044	0.995536	0.992126	0.000009	...	False	False	

5 rows × 61 columns

Label Encoding

Binary Labels

```
# changing attack labels into two categories 'normal' and 'abnormal'
bin_label = pd.DataFrame(data.label.map(lambda x:'normal' if x==0 else 'abnormal'))


# creating a dataframe with binary labels (normal,abnormal)
bin_data = data.copy()
bin_data['label'] = bin_label


# label encoding (0,1) binary labels
le1 = preprocessing.LabelEncoder()
enc_label = bin_label.apply(le1.fit_transform)
bin_data['label'] = enc_label


le1.classes_
```

```
array(['abnormal', 'normal'], dtype=object)
```

```
np.save("le1_classes.npy",le1.classes_,allow_pickle=True)
```

Multi-class Labels

```
# one-hot-encoding attack label
multi_data = data.copy()
multi_label = pd.DataFrame(multi_data.attack_cat)


multi_data = pd.get_dummies(multi_data,columns=['attack_cat'])


# label encoding (0,1,2,3,4,5,6,7,8) multi-class labels
le2 = preprocessing.LabelEncoder()
enc_label = multi_label.apply(le2.fit_transform)
multi_data['label'] = enc_label


le2.classes_
```

```
array(['Analysis', 'Backdoor', 'DoS', 'Exploits', 'Fuzzers', 'Generic',
       'Normal', 'Reconnaissance', 'Worms'], dtype=object)
```

```
np.save("le2_classes.npy",le2.classes_,allow_pickle=True)
```

Correlation between features of dataset

[ ] ↳ 6 cells hidden

Feature Selection

Binary Labels

```
# finding the attributes which have more than 0.3 correlation with encoded attack label attribute
corr_ybin = abs(corr_bin['label'])
highest_corr_bin = corr_ybin[corr_ybin >0.3]
highest_corr_bin.sort_values(ascending=True)
```

	label
sload	0.334562
dload	0.343910
rate	0.344535
ct_src_ltm	0.368486
ct_dst_ltm	0.387358
ct_src_dport_ltm	0.444874
ct_srv_dst	0.459984
ct_srv_src	0.463153
ct_dst_src_ltm	0.463735
ct_dst_sport_ltm	0.497234
sttl	0.707337
ct_state_ttl	0.801403
label	1.000000

dtype: float64

```
# selecting attributes found by using pearson correlation coefficient
bin_cols = highest_corr_bin.index
bin_cols
```

```
Index(['rate', 'sttl', 'sload', 'dload', 'ct_srv_src', 'ct_state_ttl',
       'ct_dst_ltm', 'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm',
       'ct_src_ltm', 'ct_srv_dst', 'label'],
      dtype='object')
```

```
# Binary labelled Dataset
bin_data = bin_data[bin_cols].copy()
bin_data
```

	rate	sttl	sload	dload	ct_srv_src	ct_state_ttl	ct_dst_ltm	ct_src_dport_ltm	ct_dst_sport_ltm	ct_dst_sr
3	0.000014	0.138393	0.000001	0.000150	0.000000	0.166667	0.02	0.00	0.000000	0.0
11	0.000043	0.138393	0.000092	0.000364	0.019608	0.166667	0.00	0.00	0.000000	0.0
15	0.500000	0.995536	0.119792	0.000000	0.000000	0.333333	0.00	0.00	0.000000	0.0
17	0.000043	0.138393	0.000007	0.000869	0.039216	0.166667	0.00	0.00	0.000000	0.0
21	0.000044	0.995536	0.000009	0.000236	0.098039	0.166667	0.00	0.00	0.000000	0.0
...	...	...	...	...	...	...	...	...	...	
175335	0.166667	0.995536	0.032986	0.000000	0.862745	0.333333	0.64	0.64	0.533333	0.8
175336	0.111111	0.995536	0.021991	0.000000	0.450980	0.333333	0.46	0.46	0.400000	0.4
175338	0.111111	0.995536	0.021991	0.000000	0.215686	0.333333	0.04	0.04	0.066667	0.2
175339	0.111111	0.995536	0.021991	0.000000	0.568627	0.333333	0.58	0.58	0.433333	0.5
175340	0.111111	0.995536	0.021991	0.000000	0.568627	0.333333	0.58	0.58	0.500000	0.5

81173 rows × 13 columns

Saving Prepared Dataset to Disk

```
bin_data.to_csv('/content/drive/MyDrive/Datasets/bin_data.csv')
```

Multi-class Labels

```
# finding the attributes which have more than 0.3 correlation with encoded attack label attribute
corr_ymulti = abs(corr_multi['label'])
highest_corr_multi = corr_ymulti[corr_ymulti >0.3]
highest_corr_multi.sort_values(ascending=True)
```

	label
swin	0.364393
dwin	0.364393
synack	0.524027
ackdat	0.570098
tcprtt	0.570205
dttl	0.646589
label	1.000000

dtype: float64

```
# selecting attributes found by using pearson correlation coefficient
multi_cols = highest_corr_multi.index
multi_cols
```

```
Index(['dttl', 'swin', 'dwin', 'tcprtt', 'synack', 'ackdat', 'label'], dtype='object')
```

```
# Multi-class labelled Dataset
multi_data = multi_data[multi_cols].copy()
```

Saving Prepared Dataset to Disk

```
multi_data.to_csv('/content/drive/MyDrive/Datasets/multi_data.csv')
```

BINARY CLASSIFICATION

K Nearest Neighbor Classifier

```
from sklearn.model_selection import train_test_split

# For Binary Classification
X = bin_data.drop('label', axis=1)  # Features
y = bin_data['label']  # Target variable


# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  # Adjust test_size and random_state a


knn_bin=KNeighborsClassifier(n_neighbors=5)
knn_bin.fit(X_train,y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
y_pred = knn_bin.predict(X_test)


print("Mean Absolute Error - " , metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error - " , metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error - " , np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score - " , metrics.explained_variance_score(y_test, y_pred)*100)
print("Accuracy - ",accuracy_score(y_test,y_pred)*100)
```

```
Mean Absolute Error -  0.018417000307976595
Mean Squared Error -  0.018417000307976595
Root Mean Squared Error -  0.1357092491614945
R2 Score -  89.86333626507398
Accuracy -  98.15829996920235
```

```
cls_report = classification_report(y_true=y_test, y_pred=y_pred, target_names=le1.classes_)
print(cls_report)
```

```
              precision    recall  f1-score   support

    abnormal       0.98      0.99      0.99     12370
      normal       0.97      0.95      0.96      3865

    accuracy                           0.98     16235
   macro avg       0.98      0.97      0.97     16235
weighted avg       0.98      0.98      0.98     16235
```

Real and Predicted Data

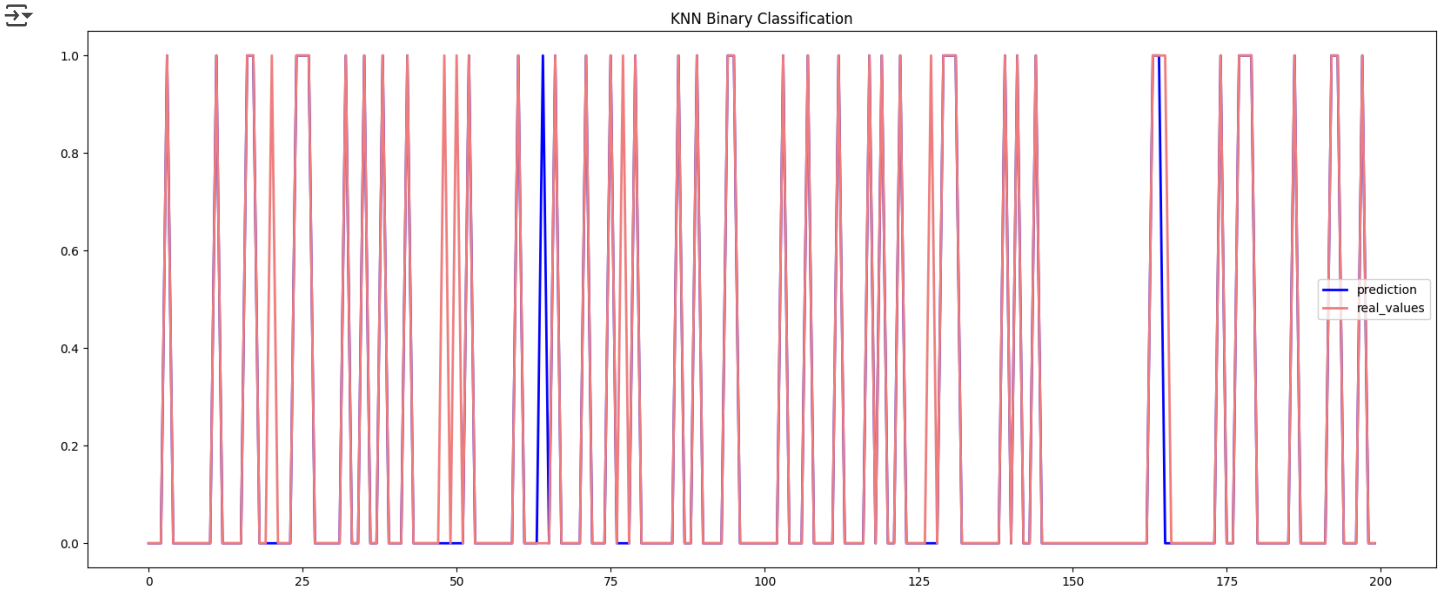
```
knn_bin_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
knn_bin_df.to_csv('/content/drive/MyDrive/predictions/knn_real_pred_bin.csv')
knn_bin_df
```

	Actual	Predicted
128512	0	0
63086	0	0
146991	0	0
38684	1	1
131913	0	0
...	...	...
32213	1	1
98044	0	0
148928	0	0
18756	1	1
38772	1	1

16235 rows × 2 columns

Plot between Real and Predicted Data

```
plt.figure(figsize=(20,8))
plt.plot(y_pred[:200], label="prediction", linewidth=2.0,color='blue')
plt.plot(y_test[:200].values, label="real_values", linewidth=2.0,color='lightcoral')
plt.legend(loc="best")
plt.title("KNN Binary Classification")
plt.savefig('/content/drive/MyDrive/plots/knn_real_pred_bin.png')
plt.show()
```



Saving Trained Model to Disk

```
pkl_filename = "/content/drive/MyDrive/models/knn_binary.pkl"
if (not path.isfile(pkl_filename)):
  # saving the trained model to disk
  with open(pkl_filename, 'wb') as file:
```

```
    pickle.dump(knn_bin, file)
  print("Saved model to disk")
else:
  print("Model already saved")
```

```
Model already saved
```

Random Forest Classifier

```
rf_bin = RandomForestClassifier(random_state=123)
rf_bin.fit(X_train,y_train)
```

```
RandomForestClassifier(random_state=123)
```

```
y_pred = rf_bin.predict(X_test)
```

```
print("Mean Absolute Error - " , metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error - " , metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error - " , np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score - " , metrics.explained_variance_score(y_test, y_pred)*100)
print("Accuracy - ",accuracy_score(y_test,y_pred)*100)
```

```
Mean Absolute Error -  0.015214043732676317
Mean Squared Error -  0.015214043732676317
Root Mean Squared Error -  0.12334522176669965
R2 Score -  91.62494914772104
Accuracy -  98.47859562673237
```

```
# Replace le1.classes_ with le2.classes_ to match the number of classes in your multi-class classification
# Original: cls_report= classification_report(y_true=y_test, y_pred=y_pred,target_names=le2.classes_)
cls_report= classification_report(y_true=y_test, y_pred=y_pred,target_names=le1.classes_) # Use le1.classes_ for binary classifi
print(cls_report)
```

```
              precision    recall  f1-score   support

    abnormal       0.99      0.99      0.99     12370
      normal       0.98      0.96      0.97      3865

    accuracy                           0.98     16235
   macro avg       0.98      0.98      0.98     16235
weighted avg       0.98      0.98      0.98     16235
```

Real and Predicted Data

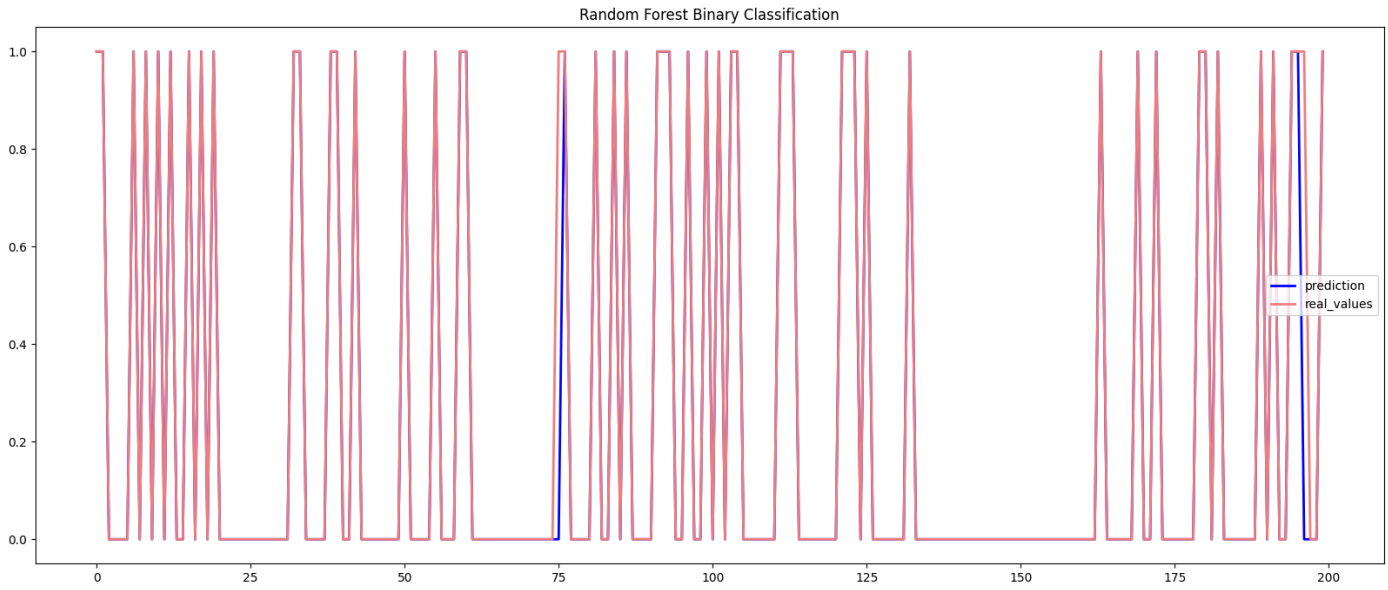
```
rf_bin_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
rf_bin_df.to_csv('/content/drive/MyDrive/predictions/rf_real_pred_bin.csv')
rf_bin_df
```

	Actual	Predicted
128512	0	0
63086	0	0
146991	0	0
38684	1	1
131913	0	0
...	...	...
32213	1	1
98044	0	0
148928	0	0
18756	1	1
38772	1	1

16235 rows × 2 columns

Plot between Real and Predicted Data

```
plt.figure(figsize=(20,8))
plt.plot(y_pred[200:400], label="prediction", linewidth=2.0,color='blue')
plt.plot(y_test[200:400].values, label="real_values", linewidth=2.0,color='lightcoral')
plt.legend(loc="best")
plt.title("Random Forest Binary Classification")
plt.savefig('/content/drive/MyDrive/plots/rf_real_pred_bin.png')
plt.show()
```



Saving Trained Model to Disk

```
pkl_filename = "/content/drive/MyDrive/models/random_forest_binary.pkl"
if (not path.isfile(pkl_filename)):
  # saving the trained model to disk
  with open(pkl_filename, 'wb') as file:
```

```
    pickle.dump(rf_bin, file)
  print("Saved model to disk")
else:
  print("Model already saved")
```

```
Model already saved
```

Decision Tree Classifier

```
dt_bin = DecisionTreeClassifier(random_state=123)
dt_bin.fit(X_train,y_train)
```

```
DecisionTreeClassifier(random_state=123)
```

```
y_pred = dt_bin.predict(X_test)
```

```
print("Mean Absolute Error - " , metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error - " , metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error - " , np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score - " , metrics.explained_variance_score(y_test, y_pred)*100)
print("Accuracy - ",accuracy_score(y_test,y_pred)*100)
```

```
Mean Absolute Error -  0.020757622420696027
Mean Squared Error -  0.020757622420696027
Root Mean Squared Error -  0.14407505828801884
R2 Score -  88.55674486849522
Accuracy -  97.9242377579304
```

```
cls_report = classification_report(y_true=y_test, y_pred=y_pred, target_names=le1.classes_)
print(cls_report)
```

```
              precision    recall  f1-score   support

      abnormal       0.99      0.99      0.99     12370
        normal       0.96      0.95      0.96      3865

      accuracy                           0.98     16235
     macro avg       0.97      0.97      0.97     16235
  weighted avg       0.98      0.98      0.98     16235
```

Real and Predicted Data

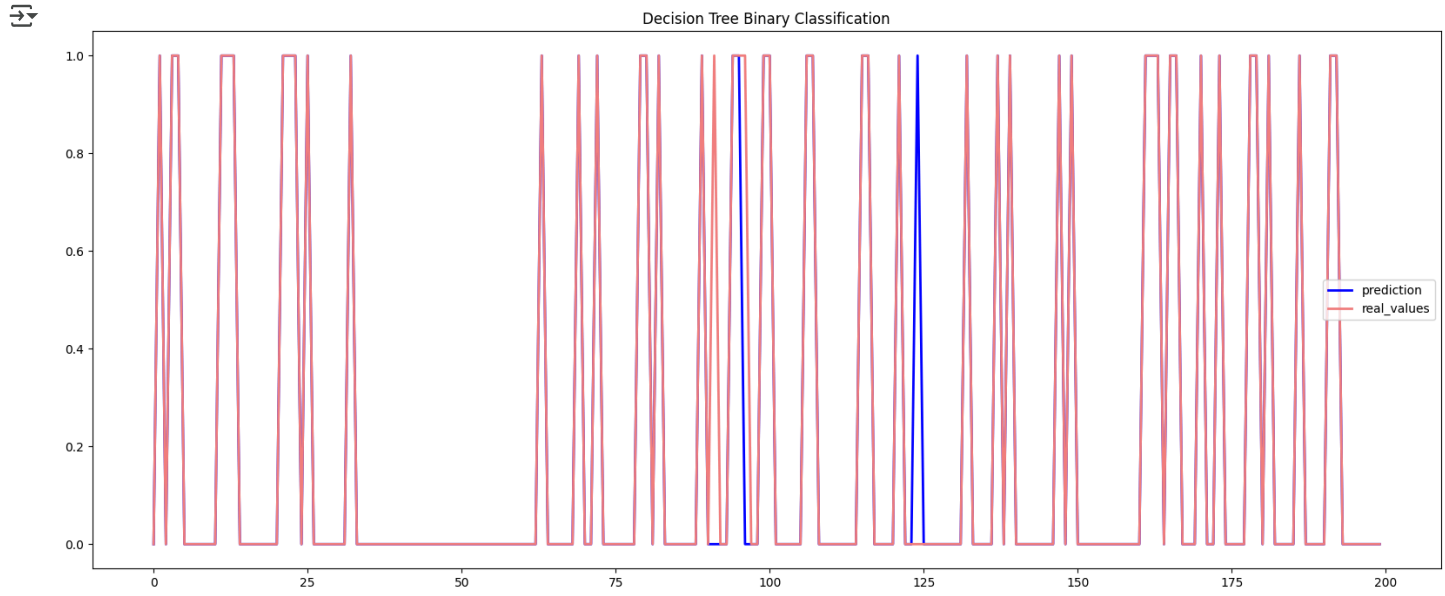
```
dt_bin_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
dt_bin_df.to_csv('/content/drive/MyDrive/predictions/dt_real_pred_bin.csv')
dt_bin_df
```

	Actual	Predicted
128512	0	0
63086	0	0
146991	0	0
38684	1	1
131913	0	0
...	...	...
32213	1	1
98044	0	0
148928	0	0
18756	1	1
38772	1	1

16235 rows × 2 columns

Plot between Real and Predicted Data

```
plt.figure(figsize=(20,8))
plt.plot(y_pred[300:500], label="prediction", linewidth=2.0,color='blue')
plt.plot(y_test[300:500].values, label="real_values", linewidth=2.0,color='lightcoral')
plt.legend(loc="best")
plt.title("Decision Tree Binary Classification")
plt.savefig('/content/drive/MyDrive/plots/dt_real_pred_bin.png')
plt.show()
```



Saving Trained Model to Disk

```
pkl_filename = "/content/drive/MyDrive/models/decision_tree_binary.pkl"
if (not path.isfile(pkl_filename)):
  # saving the trained model to disk
  with open(pkl_filename, 'wb') as file:
    pickle.dump(dt_bin, file)
  print("Saved model to disk")
else:
  print("Model already saved")
```

```
Model already saved
```

MULTI-CLASS CLASSIFICATION

K Nearest Neighbor Classifier

```
knn_multi = KNeighborsClassifier(n_neighbors=5)
knn_multi.fit(X_train,y_train)
```

```
KNeighborsClassifier()
```

```
y_pred = knn_multi.predict(X_test)


print("Mean Absolute Error - " , metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error - " , metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error - " , np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score - " , metrics.explained_variance_score(y_test, y_pred)*100)
print("Accuracy - ",accuracy_score(y_test,y_pred)*100)
```

```
Mean Absolute Error -  0.018417000307976595
Mean Squared Error -  0.018417000307976595
Root Mean Squared Error -  0.1357092491614945
R2 Score -  89.86333626507398
Accuracy -  98.15829996920235
```

```
# Split the multi-class data into training and testing sets
X = multi_data.drop('label', axis=1)  # Features
y = multi_data['label']  # Target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  # Adjust test_size and random_state a

# Train the K Nearest Neighbors model
knn_multi = KNeighborsClassifier(n_neighbors=5)
knn_multi.fit(X_train, y_train)

# Make predictions
y_pred = knn_multi.predict(X_test)

# Generate the classification report with the correct target names
print(classification_report(y_test, y_pred, target_names=le2.classes_))
```

```
                precision    recall  f1-score   support

      Analysis       0.26      0.22      0.24       126
      Backdoor       0.00      0.00      0.00        17
           DoS       0.11      0.04      0.06       367
      Exploits       0.69      0.91      0.78      3309
       Fuzzers       0.17      0.04      0.06       355
       Generic       0.98      0.99      0.99      7828
        Normal       0.98      0.91      0.94      3865
Reconnaissance       0.02      0.00      0.01       339
         Worms       0.00      0.00      0.00        29

      accuracy                           0.88     16235
     macro avg       0.36      0.35      0.34     16235
  weighted avg       0.86      0.88      0.86     16235

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Real and Predicted Data

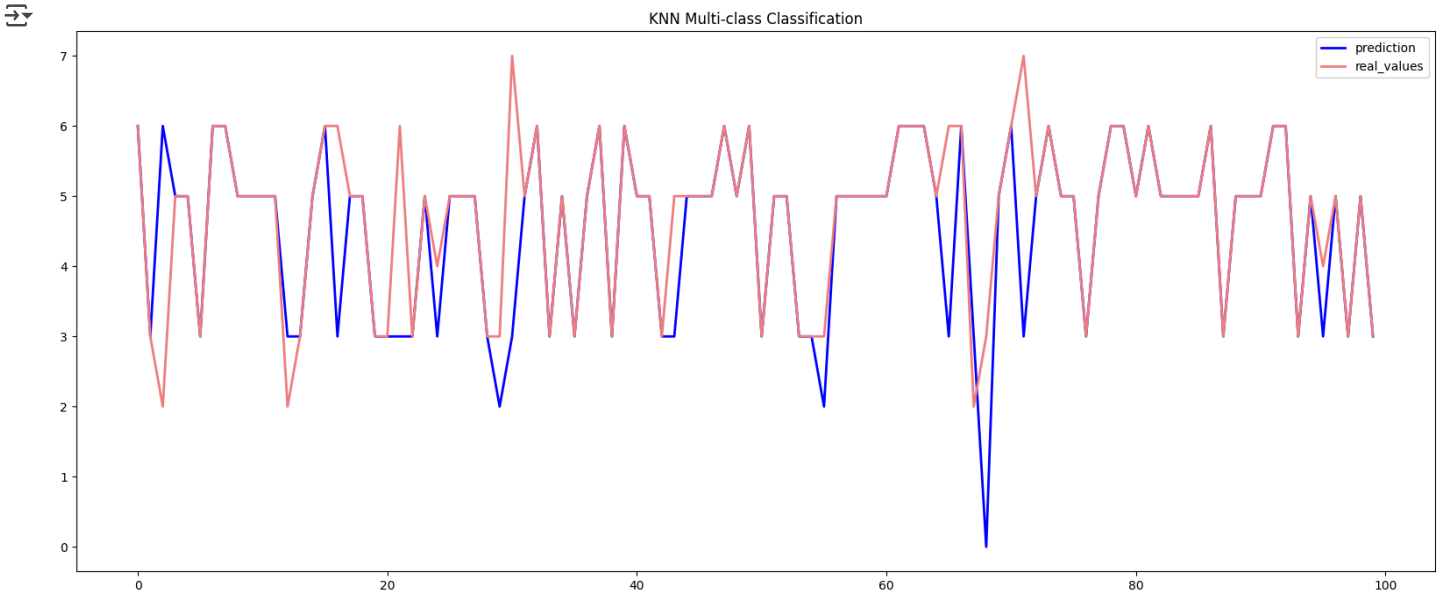
```
knn_multi_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
knn_multi_df.to_csv('/content/drive/MyDrive/predictions/knn_real_pred_multi.csv')
knn_multi_df
```

	Actual	Predicted
128512	5	5
63086	3	0
146991	5	5
38684	6	6
131913	5	5
...	...	...
32213	6	6
98044	2	3
148928	5	5
18756	6	6
38772	6	6

16235 rows × 2 columns

Plot between Real and Predicted Data

```
plt.figure(figsize=(20,8))
plt.plot(y_pred[400:500], label="prediction", linewidth=2.0,color='blue')
plt.plot(y_test[400:500].values, label="real_values", linewidth=2.0,color='lightcoral')
plt.legend(loc="best")
plt.title("KNN Multi-class Classification")
plt.savefig('/content/drive/MyDrive/plots/knn_real_pred_multi.png')
plt.show()
```



Saving Trained Model to Disk

```
pkl_filename = "/content/drive/MyDrive/models/knn_multi.pkl"
if (not path.isfile(pkl_filename)):
  # saving the trained model to disk
```

```
  with open(pkl_filename, 'wb') as file:
    pickle.dump(knn_multi, file)
  print("Saved model to disk")
else:
  print("Model already saved")
```

```
Model already saved
```

Random Forest Classifier

```
rf_multi = RandomForestClassifier(random_state=50)
rf_multi.fit(X_train,y_train)
```

```
RandomForestClassifier(random_state=50)
```

```
y_pred = rf_multi.predict(X_test)
```

```
print("Mean Absolute Error - " , metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error - " , metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error - " , np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score - " , metrics.explained_variance_score(y_test, y_pred)*100)
print("Accuracy - ",accuracy_score(y_test,y_pred)*100)
```

```
Mean Absolute Error -  0.263935940868494
Mean Squared Error -  0.8128734216199569
Root Mean Squared Error -  0.9015949321174986
R2 Score -  47.136132157360514
Accuracy -  88.72189713581767
```

```
print(classification_report(y_test, y_pred,target_names=le2.classes_))
```

```
                precision    recall  f1-score   support

          Analysis       0.67      0.44      0.53       126
          Backdoor       0.12      0.06      0.08        17
               DoS       0.17      0.06      0.09       367
          Exploits       0.70      0.90      0.79      3309
           Fuzzers       0.29      0.09      0.14       355
           Generic       0.98      0.99      0.99      7828
            Normal       0.97      0.92      0.94      3865
    Reconnaissance       0.12      0.03      0.05       339
             Worms       0.00      0.00      0.00        29

          accuracy                           0.89     16235
         macro avg       0.45      0.39      0.40     16235
      weighted avg       0.86      0.89      0.87     16235
```

Real and Predicted Data

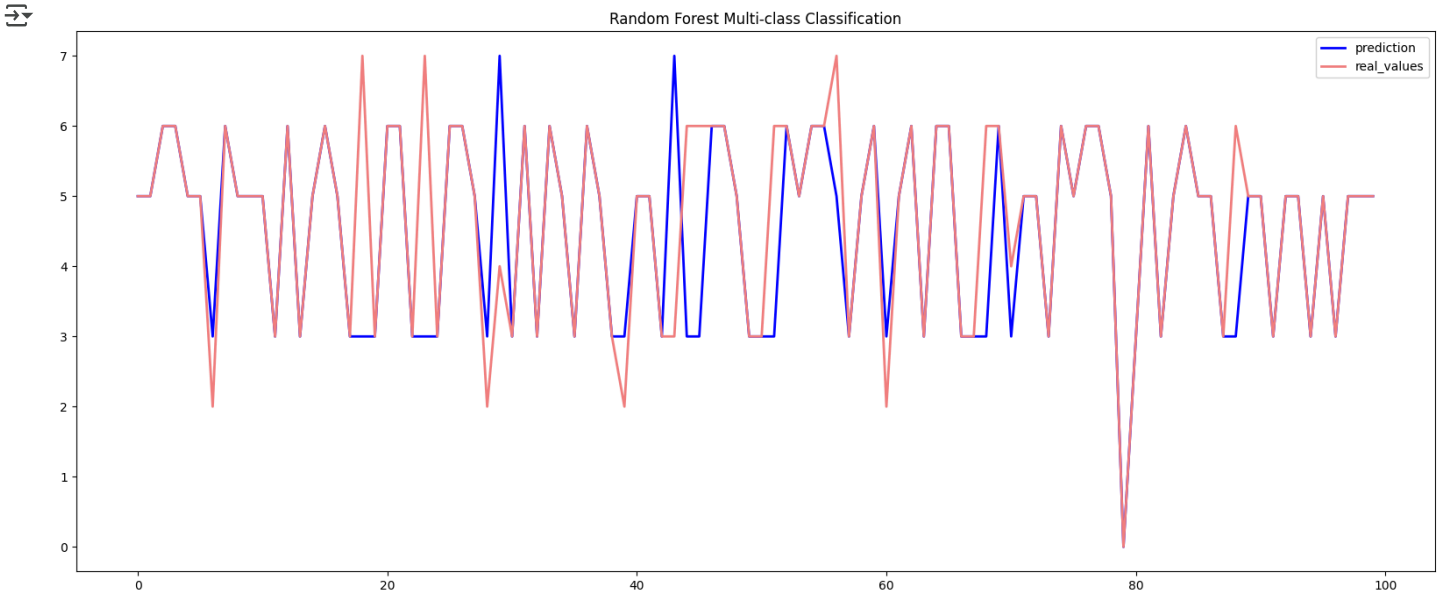
```
rf_multi_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
rf_multi_df.to_csv('/content/drive/MyDrive/predictions/rf_real_pred_multi.csv')
rf_multi_df
```

	Actual	Predicted
128512	5	5
63086	3	3
146991	5	5
38684	6	6
131913	5	5
...	...	...
32213	6	6
98044	2	3
148928	5	5
18756	6	6
38772	6	6

16235 rows × 2 columns

Plot between Real and Predicted Data

```
plt.figure(figsize=(20,8))
plt.plot(y_pred[500:600], label="prediction", linewidth=2.0,color='blue')
plt.plot(y_test[500:600].values, label="real_values", linewidth=2.0,color='lightcoral')
plt.legend(loc="best")
plt.title("Random Forest Multi-class Classification")
plt.savefig('/content/drive/MyDrive/plots/rf_real_pred_multi.png')
plt.show()
```



Saving Trained Model to Disk

```
pkl_filename = "/content/drive/MyDrive/models/random_forest_multi.pkl
if (not path.isfile(pkl_filename)):
  # saving the trained model to disk
```